

# Experiment 08

## Views & Materialized Views in PostgreSQL

Name: Deven

UID: 24BCF10060

### Aim

To study and implement PostgreSQL **Views** and **Materialized Views**, including exposing a restricted/computed column set through a view, controlling access to a view using database roles and privileges, and comparing the query performance of a live analytical query against a materialized view using `EXPLAIN ANALYZE`.

### Part A – Views in PostgreSQL

A *view* is a stored `SELECT` query that behaves like a virtual table. A view does not store data physically; every time it is queried, PostgreSQL re-executes the underlying query against the base tables. Views are commonly used to simplify complex joins, present a controlled and computed subset of columns to different users, and enforce column-level security by hiding sensitive base-table columns.

### Question 1 – Order Report View (Medium)

A company maintains customer information, product information, and sales order information in three separate tables: `customer_master`, `product_catalog`, and `sales_orders`.

The company wants to provide an order report to an external client. The report must contain the Order ID, Order Date, Customer Name, Product Name, and Final Amount. For every order, the amounts are calculated as:

$$\text{Gross Amount} = \text{Unit Price} \times \text{Quantity}$$

$$\text{Discount Amount} = \text{Gross Amount} \times \frac{\text{Discount Percentage}}{100}$$

$$\text{Final Amount} = \text{Gross Amount} - \text{Discount Amount}$$

Create a view using the three tables to generate this report. The view must expose only the five required columns and must *not* expose the customer's phone/email, the product's brand/unit price, or the order's quantity/discount percentage.

The company also wants to provide access to an external client. Create a PostgreSQL user named `client.user` and grant this user only `SELECT` permission on the view. The client must not have direct access to the three original tables and must not be allowed to modify the view's data.

### Given Data

```

1 CREATE TABLE customer_master
2 (
3     customer_id VARCHAR(5) PRIMARY KEY,
4     full_name   VARCHAR(50),
5     phone       VARCHAR(15),
6     email       VARCHAR(50),
7     city        VARCHAR(50)

```

```
8 );
9
10 CREATE TABLE product_catalog
11 (
12     product_id    VARCHAR(5) PRIMARY KEY,
13     product_name  VARCHAR(50),
14     brand         VARCHAR(30),
15     unit_price    NUMERIC(10,2)
16 );
17
18 CREATE TABLE sales_orders
19 (
20     order_id      INT PRIMARY KEY,
21     product_id    VARCHAR(5),
22     quantity      INT,
23     customer_id   VARCHAR(5),
24     discount_percent NUMERIC(5,2),
25     order_date    DATE,
26     FOREIGN KEY(product_id) REFERENCES product_catalog(product_id),
27     FOREIGN KEY(customer_id) REFERENCES customer_master(customer_id)
28 );
29
30 INSERT INTO customer_master VALUES
31 ('C1','Amit Sharma','9876500001','amit.sharma@example.com','Delhi'),
32 ('C2','Priya Verma','9876500002','priya.verma@example.com','Mumbai'),
33 ('C3','Ravi Kumar','9876500003','ravi.kumar@example.com','Bangalore')
34 ,
35 ('C4','Neha Singh','9876500004','neha.singh@example.com','Kolkata'),
36 ('C5','Arjun Mehta','9876500005','arjun.mehta@example.com','Hyderabad')
37 );
38
39 INSERT INTO product_catalog VALUES
40 ('P1','Smartphone X100','TechNova',25000),
41 ('P2','Laptop Pro 15','TechNova',65000),
42 ('P3','Wireless Earbuds','SoundWave',5000),
43 ('P4','Smartwatch Fit','FitTrack',30000),
44 ('P5','Gaming Console','PlayZone',45000);
45
46 INSERT INTO sales_orders VALUES
47 (1,'P1',2,'C1',5,'2025-09-01'),
48 (2,'P2',1,'C2',10,'2025-09-02'),
49 (3,'P3',3,'C3',0,'2025-09-03'),
50 (4,'P4',1,'C4',8,'2025-09-04'),
51 (5,'P5',1,'C5',12,'2025-09-05'),
52 (6,'P1',1,'C2',5,'2025-09-06'),
53 (7,'P3',2,'C1',0,'2025-09-07');
```



Answer

*Step 1 – Create the restricted, computed-column view*

```
1 CREATE VIEW vw_order_report AS
2
3 SELECT
4     s.order_id,
5     s.order_date,
6     c.full_name AS customer_name,
7     p.product_name,
8
9     ROUND(
10         (p.unit_price * s.quantity)
11         - (p.unit_price * s.quantity * s.discount_percent / 100)
12     , 2) AS final_amount
13
14 FROM sales_orders s
15
16 JOIN customer_master c
17 ON s.customer_id = c.customer_id
18
19 JOIN product_catalog p
20 ON s.product_id = p.product_id;
21
22 SELECT * FROM vw_order_report
23 ORDER BY order_id;
```

*Step 2 – Create the restricted client role*

```
1 CREATE USER client_user WITH PASSWORD 'Client@123';
2
3 -- Explicitly ensure the client has no access to the base tables
4 REVOKE ALL ON customer_master, product_catalog, sales_orders
5 FROM client_user;
6
7 -- Grant read-only access to the view only
8 GRANT SELECT ON vw_order_report TO client_user;
9
10 -- A join-based view is not updatable by default, but this is added
11 -- as an explicit safeguard so the client can never modify view data
12 REVOKE INSERT, UPDATE, DELETE ON vw_order_report FROM client_user;
```

Output

| order_id | order_date | customer_name | product_name     | final_amount |
|----------|------------|---------------|------------------|--------------|
| 1        | 2025-09-01 | Amit Sharma   | Smartphone X100  | 47500.00     |
| 2        | 2025-09-02 | Priya Verma   | Laptop Pro 15    | 58500.00     |
| 3        | 2025-09-03 | Ravi Kumar    | Wireless Earbuds | 15000.00     |
| 4        | 2025-09-04 | Neha Singh    | Smartwatch Fit   | 27600.00     |
| 5        | 2025-09-05 | Arjun Mehta   | Gaming Console   | 39600.00     |
| 6        | 2025-09-06 | Priya Verma   | Smartphone X100  | 23750.00     |
| 7        | 2025-09-07 | Amit Sharma   | Wireless Earbuds | 10000.00     |

## Part B – Materialized Views in PostgreSQL

Unlike a regular view, a *materialized view* physically stores the result of its defining query on disk at the time it is created. This avoids repeating expensive joins, grouping, and aggregation on every access, which is beneficial for analytical reports that do not need to reflect the base tables in real time. Because the data is a stored snapshot, a materialized view must be explicitly synchronised with the base tables using `REFRESH MATERIALIZED VIEW`; until it is refreshed, it continues to return the data that existed at the time of the last refresh.

### Question 2 – Category Performance Materialized View (Hard)

An e-commerce company wants to regularly analyse the performance of its product categories. The database needs to perform multiple joins, `GROUP BY`, aggregate functions, `DISTINCT` counting, calculated values, and `CASE` statements to build this report. Since the same report is generated frequently, executing the full analytical query every time increases database workload.

Create a PostgreSQL materialized view named `mv_category_sales`. Also demonstrate that when new order data is inserted, the materialized view does *not* automatically reflect the change and must be refreshed. Finally, use `EXPLAIN ANALYZE` to compare the performance of the original analytical query with the materialized view query.

#### Business Rules

| Requirement            | Rule                                     |
|------------------------|------------------------------------------|
| Total Products Sold    | <code>SUM(quantity)</code>               |
| Total Orders           | <code>COUNT(DISTINCT order_id)</code>    |
| Unique Customers       | <code>COUNT(DISTINCT customer_id)</code> |
| Total Revenue          | <code>SUM(quantity × unit_price)</code>  |
| Avg. Revenue per Order | Total Revenue ÷ Total Orders             |
| Excellent              | Revenue ≥ 10,000                         |
| Good                   | Revenue ≥ 6,000 and < 10,000             |
| Average                | Revenue ≥ 3,000 and < 6,000              |
| Low                    | Revenue < 3,000                          |

#### Given Data

```

1 CREATE TABLE category_master
2 (
3     category_id    VARCHAR(5) PRIMARY KEY,
4     category_name  VARCHAR(50)
5 );
6
7 CREATE TABLE product_master
8 (
9     product_id     VARCHAR(5) PRIMARY KEY,
10    product_name    VARCHAR(50),
11    category_id     VARCHAR(5),
12    FOREIGN KEY(category_id) REFERENCES category_master(category_id)
13 );
14
15 CREATE TABLE customer_master
16 (

```

```
17     customer_id    VARCHAR(5) PRIMARY KEY,
18     customer_name  VARCHAR(50)
19 );
20
21 CREATE TABLE order_details
22 (
23     order_id       VARCHAR(6) PRIMARY KEY,
24     customer_id    VARCHAR(5),
25     product_id     VARCHAR(5),
26     quantity       INT,
27     unit_price     NUMERIC(10,2),
28     FOREIGN KEY(customer_id) REFERENCES customer_master(customer_id),
29     FOREIGN KEY(product_id) REFERENCES product_master(product_id)
30 );
31
32 INSERT INTO category_master VALUES
33 ('C1','Electronics'),
34 ('C2','Clothing'),
35 ('C3','Books');
36
37 INSERT INTO product_master VALUES
38 ('P101','Laptop','C1'),
39 ('P102','Headphones','C1'),
40 ('P201','Jacket','C2'),
41 ('P202','Shoes','C2'),
42 ('P301','SQL Book','C3');
43
44 INSERT INTO customer_master VALUES
45 ('U1','Aman'),
46 ('U2','Priya'),
47 ('U3','Rahul'),
48 ('U4','Simran');
49
50 INSERT INTO order_details VALUES
51 ('O101','U1','P101',1,5000),
52 ('O102','U2','P102',2,1000),
53 ('O103','U1','P201',1,3000),
54 ('O104','U3','P202',2,2000),
55 ('O105','U4','P301',3,500),
56 ('O106','U2','P101',1,5000),
57 ('O107','U3','P201',1,3000),
58 ('O108','U4','P102',1,1000);
```



## Answer

### Step 1 – Category-wise sales query (with EXPLAIN ANALYZE)

```

1  EXPLAIN ANALYZE
2  SELECT
3  cm.category_id,
4  cm.category_name,
5  SUM(od.quantity)           AS total_products_sold,
6  COUNT(DISTINCT od.order_id) AS total_orders,
7  COUNT(DISTINCT od.customer_id) AS unique_customers,
8  SUM(od.quantity * od.unit_price) AS total_revenue,
9
10 ROUND(
11     SUM(od.quantity * od.unit_price)::NUMERIC
12     / COUNT(DISTINCT od.order_id)
13     , 2) AS avg_revenue_per_order,
14
15 CASE
16     WHEN SUM(od.quantity * od.unit_price) >= 10000 THEN 'Excellent'
17     WHEN SUM(od.quantity * od.unit_price) >= 6000  THEN 'Good'
18     WHEN SUM(od.quantity * od.unit_price) >= 3000  THEN 'Average'
19     ELSE 'Low'
20 END AS category_performance
21
22 FROM category_master cm
23
24 JOIN product_master pm
25 ON pm.category_id = cm.category_id
26
27 JOIN order_details od
28 ON od.product_id = pm.product_id
29
30 GROUP BY cm.category_id, cm.category_name
31
32 ORDER BY cm.category_id;

```

### Sample Query Plan

```

GroupAggregate (cost=1.28..1.51 rows=3 width=96) (actual time=0.061..0.079 rows=3
 loops=1)
  Group Key: cm.category_id
  -> Sort (cost=1.28..1.30 rows=8 width=68) (actual time=0.052..0.055 rows=8 loops=1)
    Sort Key: cm.category_id
    Sort Method: quicksort  Memory: 25kB
    -> Hash Join (cost=1.11..1.16 rows=8 width=68) (actual time=0.035..0.041 rows
      =8 loops=1)
      Hash Cond: (od.product_id = pm.product_id)
      -> Seq Scan on order_details od (cost=0.00..1.08 rows=8 width=19)
      -> Hash (cost=1.05..1.05 rows=5 width=57)
        -> Hash Join (cost=1.02..1.05 rows=5 width=57)
          Hash Cond: (pm.category_id = cm.category_id)
          -> Seq Scan on product_master pm
          -> Hash -> Seq Scan on category_master cm

Planning Time: 0.312 ms
Execution Time: 0.146 ms

```



### Step 2 – Create the materialized view

```

1 CREATE MATERIALIZED VIEW mv_category_sales AS
2
3 SELECT
4   cm.category_id,
5   cm.category_name,
6   SUM(od.quantity)           AS total_products_sold,
7   COUNT(DISTINCT od.order_id) AS total_orders,
8   COUNT(DISTINCT od.customer_id) AS unique_customers,
9   SUM(od.quantity * od.unit_price) AS total_revenue,
10
11  ROUND(
12    SUM(od.quantity * od.unit_price)::NUMERIC
13    / COUNT(DISTINCT od.order_id)
14    , 2) AS avg_revenue_per_order,
15
16  CASE
17    WHEN SUM(od.quantity * od.unit_price) >= 10000 THEN 'Excellent'
18    WHEN SUM(od.quantity * od.unit_price) >= 6000  THEN 'Good'
19    WHEN SUM(od.quantity * od.unit_price) >= 3000  THEN 'Average'
20    ELSE 'Low'
21  END AS category_performance
22
23 FROM category_master cm
24
25 JOIN product_master pm
26 ON pm.category_id = cm.category_id
27
28 JOIN order_details od
29 ON od.product_id = pm.product_id
30
31 GROUP BY cm.category_id, cm.category_name
32
33 WITH DATA;

```

### Step 3 – Display data from the materialized view

```

1 SELECT * FROM mv_category_sales
2 ORDER BY category_id;

```

### Output

| cat.id | category    | sold | orders | custs | revenue | avg/order | perf.     |
|--------|-------------|------|--------|-------|---------|-----------|-----------|
| C1     | Electronics | 5    | 4      | 3     | 13000   | 3250.00   | Excellent |
| C2     | Clothing    | 4    | 3      | 2     | 10000   | 3333.33   | Excellent |
| C3     | Books       | 3    | 1      | 1     | 1500    | 1500.00   | Low       |



#### Step 4 – EXPLAIN ANALYZE on the materialized view query

```
1 EXPLAIN ANALYZE
2 SELECT * FROM mv_category_sales
3 ORDER BY category_id;
```

```
Sort (cost=1.05..1.06 rows=3 width=96) (actual time=0.014..0.016 rows=3 loops=1)
  Sort Key: category_id
  Sort Method: quicksort Memory: 25kB
  -> Seq Scan on mv_category_sales (cost=0.00..1.03 rows=3 width=96)
      (actual time=0.006..0.008 rows=3 loops=1)
Planning Time: 0.089 ms
Execution Time: 0.028 ms
```

#### Comparison

The base analytical query must perform two hash joins, a GROUP BY aggregation, and multiple DISTINCT counts every time it runs. The materialized-view query, in contrast, is a single sequential scan over an already-computed result set, so its plan is far simpler and its execution time is consistently lower. On this small sample dataset the absolute difference is a fraction of a millisecond, but on a production-sized `order_details` table the saving becomes substantial, which is the main motivation for using a materialized view for a frequently-run report.

#### Step 5 – Insert new order data and observe stale results

```
1 INSERT INTO order_details VALUES
2 ('0109', 'U1', 'P202', 2, 2000);
3
4 -- Querying the materialized view immediately still returns
5 -- the OLD (stale) data, since it has not been refreshed yet
6 SELECT * FROM mv_category_sales
7 WHERE category_id = 'C2';
```

#### Output (before refresh – stale)

| cat_id | category | sold | orders | custs | revenue | avg/order | perf.     |
|--------|----------|------|--------|-------|---------|-----------|-----------|
| C2     | Clothing | 4    | 3      | 2     | 10000   | 3333.33   | Excellent |

#### Step 6 – Refresh the materialized view

```
1 REFRESH MATERIALIZED VIEW mv_category_sales;
2
3 SELECT * FROM mv_category_sales
4 WHERE category_id = 'C2';
```

#### Output (after refresh – updated)

| cat_id | category | sold | orders | custs | revenue | avg/order | perf.     |
|--------|----------|------|--------|-------|---------|-----------|-----------|
| C2     | Clothing | 6    | 4      | 2     | 14000   | 3500.00   | Excellent |

The REFRESH MATERIALIZED VIEW statement re-executed the defining query and replaced the stored snapshot, so the new order (0109) is now correctly reflected: total products sold increased from 4 to 6, total orders from 3 to 4, and total revenue from 10,000 to 14,000.

## Learning Outcome

After completing this experiment, the following learning outcomes were achieved.

- Understood how a **VIEW** exposes a restricted, computed set of columns while hiding sensitive base-table data.
- Learned to manage database access control by creating a PostgreSQL role and granting it column/row-restricted, read-only privileges through **GRANT** and **REVOKE**.
- Implemented multi-table joins with derived/calculated columns (gross amount, discount amount, final amount) inside a view definition.
- Understood the difference between a regular view (always live) and a **MATERIALIZED VIEW** (a stored, physical snapshot of query results).
- Learned to build an analytical report using multiple joins, **GROUP BY**, **DISTINCT** counts, and business-rule classification with **CASE**.
- Used **EXPLAIN ANALYZE** to compare the query execution plan and performance of a live analytical query against a materialized view.
- Observed that a materialized view becomes stale after base-table changes and must be explicitly synchronised using **REFRESH MATERIALIZED VIEW**.